PROJECT DOCUMENTATION

LearnMate - AI assistant for smarter learning and study support.

INTRODUCTION: LearnMate is an Artificial Intelligence (AI)-based learning assistant designed to help students improve their learning experience and academic performance. It provides personalized study support, answers questions, explains difficult concepts, summarizes study materials, and helps students prepare for exams. LearnMate makes learning easier, faster, and more effective by providing instant assistance based on students' learning needs

Team ID: SWTID-2026-3908

Team leader:

NAME: S.INBHASHREE

MAIL ID:iibashree@gmail.com

Team Members:

NAME: A.INDHUMATHY

MAIL ID:indhu2332007@gmail.com

NAME: K.JANANI

MAIL ID :janani63829021@gmail.com

NAME: P. JAYASRI

MAIL ID: jayajaysir@gmail.com

LearnMate - AI assistant for smarter learning and study support.

Description

AI StudyBuddy is an AI-powered learning assistance platform developed to simplify the study process for students by leveraging modern web technologies and Generative AI. The application provides an intelligent backend system that enables students to upload study materials, generate concise summaries, create flashcards, produce quizzes, and receive personalized study plans through AI-generated responses. The backend is developed using **Node.js** and **Express.js**, following a RESTful API architecture that ensures scalability, maintainability, and secure communication between the frontend and backend services. **MongoDB** serves as the primary NoSQL database, while **Mongoose ODM** provides schema validation, data modeling, and efficient database interactions. To secure user information and protected resources, the application implements **JWT (JSON Web Token)** based authentication along with **Role-Based Access Control (RBAC).** Users are authenticated before accessing AI-powered features, ensuring that only authorized users can utilize premium learning services. Passwords are securely encrypted using **bcryptjs**, preventing unauthorized access to user credentials.

One of the key highlights of AI StudyBuddy is its integration with the **Google Gemini AI API**, which enables the application to perform advanced natural language processing tasks. Instead of relying on predefined templates, the backend dynamically communicates with the Gemini model to generate high-quality educational content based on the study material provided by the user.

Scenario-Based Case Study

Background

Rahul is a second-year engineering student preparing for multiple semester examinations while simultaneously learning new technical skills for internships and placement opportunities. His study materials are scattered across lecture notes, PDFs, textbooks, online articles, and handwritten notes, making it difficult to organize and revise effectively. Due to the large volume of content and limited preparation time, Rahul often struggles to identify important concepts, create revision notes, and evaluate his understanding before examinations.

Traditional study methods require significant manual effort to summarize lengthy materials, prepare practice questions, and create revision schedules. This process is repetitive, time-consuming, and often results in inconsistent learning outcomes.

Problem

Rahul encounters several challenges during his preparation:

- Difficulty understanding lengthy study materials within limited time.
- Manual preparation of notes consumes a significant amount of study time.
- Creating flashcards and revision questions is repetitive and labor-intensive.
- Lack of personalized study plans based on available time and examination schedules.
- No centralized platform to securely store AI-generated learning resources.
- Difficulty assessing knowledge through self-generated quizzes.
- Switching between multiple applications for note-taking, quizzes, and scheduling reduces productivity.

Solution

The **AI StudyBuddy** backend provides a centralized, AI-powered learning platform that automates several academic tasks through intelligent content generation. The system enables authenticated users to upload or enter study materials and leverage the **Google Gemini AI API** to generate educational resources automatically.

The backend processes incoming requests through secure RESTful APIs and performs the following operations:

- Authenticates users using JWT-based security.
- Stores user information and study resources in MongoDB.
- Sends study content to the Google Gemini AI model through dedicated AI service modules.
- Generates concise summaries for faster revision.
- Creates flashcards for active recall learning.
- Produces multiple-choice quizzes for self-assessment.
- Generates personalized study plans based on user preferences and examination timelines.
- Stores generated learning resources for future retrieval and continuous learning.

By separating authentication, business logic, database operations, and AI processing into modular components, the backend ensures maintainability, scalability, and efficient request handling.

Usage

Student Usage

Students register and log in to the application before accessing AI-powered learning features. After authentication, they can upload study material or provide textual content and choose the

desired AI functionality, such as generating summaries, quizzes, flashcards, or personalized study plans. The generated content can be reviewed immediately and stored for future reference.

Administrator Usage

Administrators manage registered users, monitor application activity, oversee system usage, and maintain platform integrity. Administrative privileges include managing user accounts, monitoring AI service utilization, reviewing system logs, and ensuring smooth backend operations.

Outcome

By integrating artificial intelligence with a secure backend architecture, AI StudyBuddy significantly improves the learning experience for students. Manual preparation time is reduced through automated content generation, allowing students to focus more on understanding concepts rather than organizing study materials.

The implementation of JWT authentication, MongoDB data management, and Google Gemini AI integration ensures that educational resources are generated securely, efficiently, and consistently. The modular backend architecture further supports future enhancements, enabling additional AI-powered learning capabilities to be integrated with minimal architectural changes.

As a result, AI StudyBuddy serves as a scalable and intelligent educational platform that enhances productivity, promotes effective revision, and supports personalized learning for students preparing for academic examinations and competitive assessments.

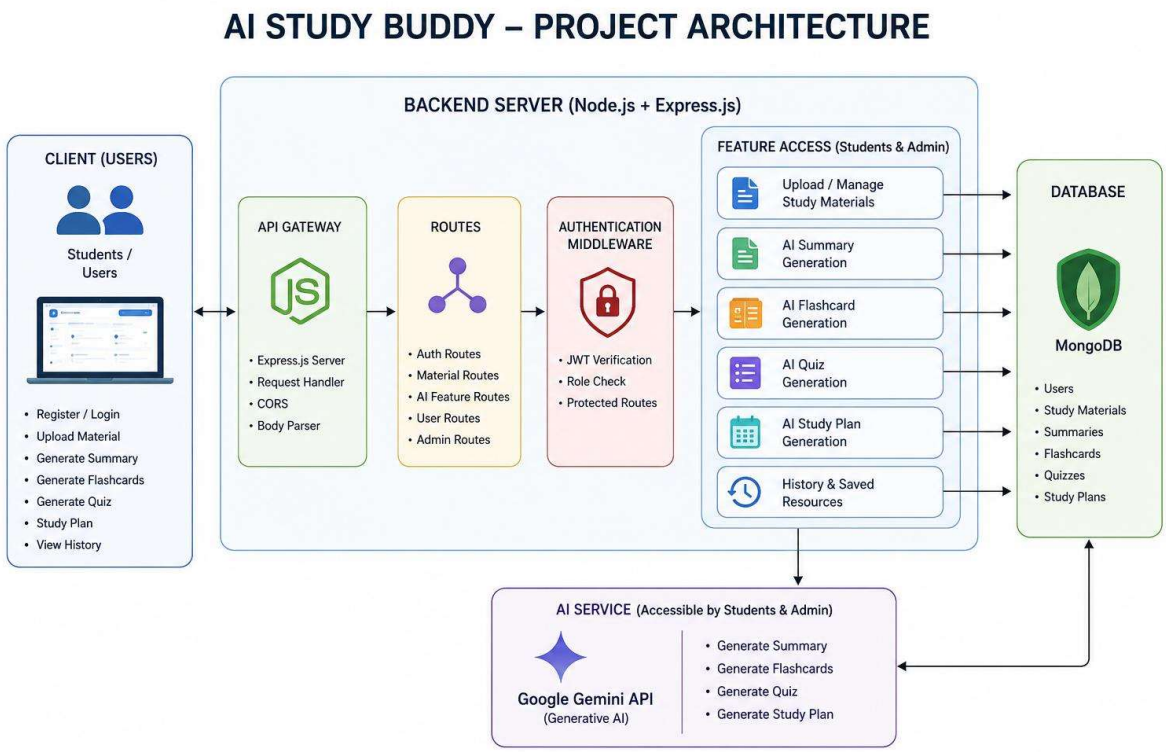
Software Requirements

- **Operating System:** Windows 10/11, macOS, or Linux (supports cross-platform operations).
- **Node.js (v16 or above):** Runtime ecosystem running server logic and routing infrastructure.
- **npm (v8 or above):** Node package manager to control operational dependencies.
- **Express.js:** Lightweight routing web framework to construct backend RESTful entry points.
- **MongoDB:** Document NoSQL engine storing users, posts, categories, comments, and analytics metrics.
- **Postman:** API verification toolkit to assert schema validation responses across administrative route shields.
- **Code Editor:** Visual Studio Code or similar IDE environment.

Hardware Requirements

- **Processor:** Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or equivalent.
- **RAM:** 8 GB minimum (16 GB recommended for concurrent instances of MongoDB, processing layers, and execution nodes).
- **Storage:** 1 GB of available disk workspace.

Epic 1: Project Architecture



The **AI StudyBuddy** backend follows a modular RESTful architecture designed to provide secure, scalable, and AI-powered learning services. The system processes client requests through Express.js, validates users using JWT authentication, interacts with the MongoDB database for data storage, and communicates with the Google Gemini AI API to generate intelligent study resources.

The architecture separates authentication, business logic, AI services, and database operations into independent modules, making the application easier to maintain, extend, and debug.

Components Definition

• **Client Application:** The frontend (React.js) allows users to register, log in, upload study material, and access AI-powered features such as summaries, quizzes, flashcards, and study plans through REST APIs.

• **Express Server:** The Express.js server receives HTTP requests, handles routing, processes middleware, and sends appropriate responses to the client.

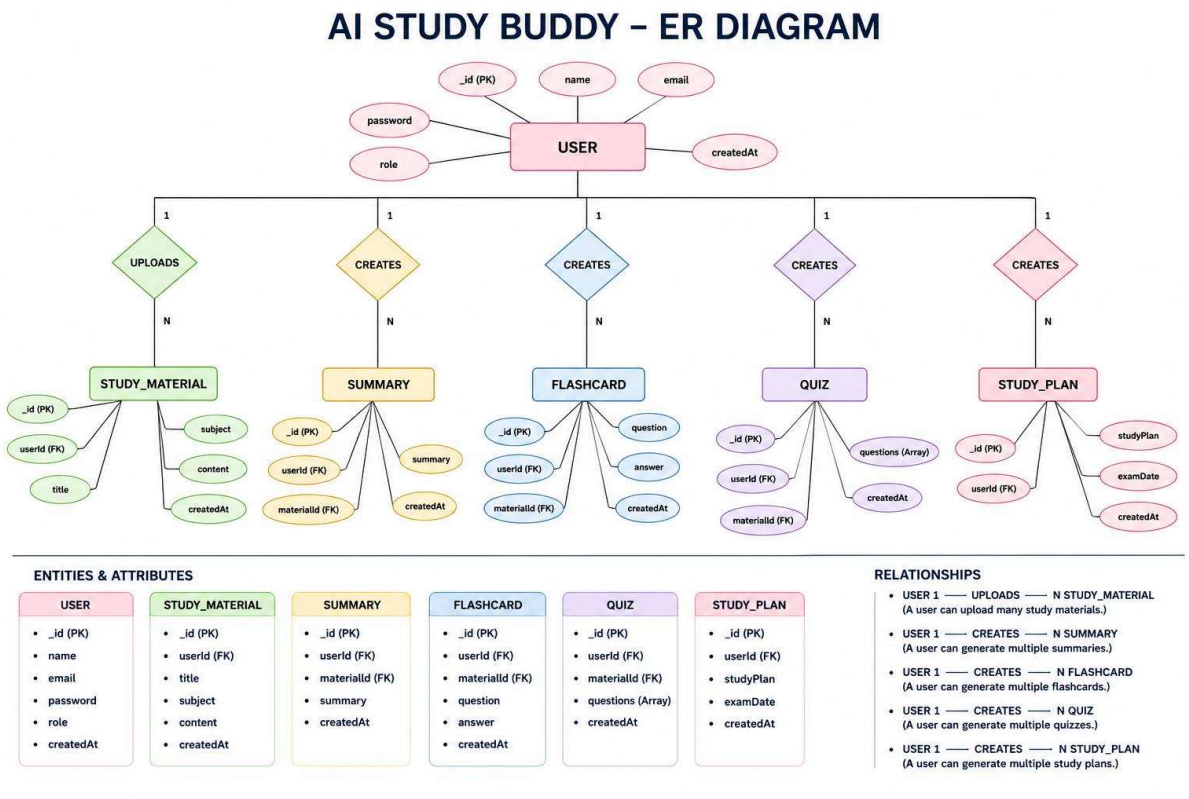
• **Authentication Middleware:** JWT-based middleware verifies user identity, protects private routes, and ensures that only authenticated users can access AI services and study resources.

• **Route Handlers:** Routes manage API endpoints related to user authentication, study materials, AI features, and administrative operations.

• **AI Service Layer:** A dedicated AI service communicates with the **Google Gemini API** to generate summaries, quizzes, flashcards, and personalized study plans based on the user's study content.

• **Database Layer:** MongoDB, together with Mongoose ODM, stores user information, uploaded study materials, and AI-generated learning resources securely.

Entity-Relationship (ER) Diagram Description



The AI StudyBuddy backend manages data through interconnected MongoDB collections that store user information, study materials, and AI-generated learning resources. Each entity is linked using logical relationships to ensure efficient data management and retrieval.

1. User Entity

Stores the details of registered users and administrators.

Attributes:

- _id (Primary Key - ObjectId)
- name (String, Required)
- email (String, Required, Unique)
- password (String, Hashed)
- role (String - Student/Admin)
- createdAt

2. Study Material Entity

Stores study content uploaded by students for AI processing.

Attributes:

- _id (Primary Key)
- userId (Foreign Key → User)
- title
- subject
- content
- createdAt

3. Summary Entity

Stores AI-generated summaries.

Attributes:

- _id (Primary Key)
- userId (Foreign Key → User)
- materialId (Foreign Key → Study Material)
- summary
- createdAt

4. Flashcard Entity

Stores AI-generated flashcards.

Attributes:

- _id (Primary Key)
- userId (Foreign Key → User)
- materialId (Foreign Key → Study Material)
- question
- answer
- createdAt

5. Quiz Entity

Stores AI-generated quizzes.

Attributes:

- _id (Primary Key)
- userId (Foreign Key → User)
- materialId (Foreign Key → Study Material)
- questions
- createdAt

6. Study Plan Entity

Stores personalized study plans generated by AI.

Attributes:

- _id (Primary Key)
- userId (Foreign Key → User)
- studyPlan
- examDate
- createdAt

Key Relationships

- **User → Study Material (One-to-Many):** A user can upload multiple study materials.
- **Study Material → Summary (One-to-One):** Each uploaded material can have one AI-generated summary.
- **Study Material → Flashcards (One-to-Many):** One study material can generate multiple flashcards.
- **Study Material → Quiz (One-to-One):** One study material can generate one quiz containing multiple questions.
- **User → Study Plan (One-to-Many):** A user can generate multiple study plans based on different subjects or examination schedules.
- **User → AI Resources (One-to-Many):** All AI-generated summaries, flashcards, quizzes, and study plans are associated with the user who generated them.

Key Features

The AI StudyBuddy backend provides intelligent learning assistance through secure and scalable REST APIs. The major features of the system are as follows:

- **Secure User Authentication:** Implements JWT-based authentication with encrypted passwords using bcrypt for secure user access.
- **Study Material Management:** Allows users to upload and manage study materials that serve as input for AI-powered learning features.
- **AI Summary Generation:** Generates concise and easy-to-understand summaries from uploaded study content using the Google Gemini API.
- **AI Flashcard Generation:** Automatically creates flashcards from study materials to improve concept revision and active recall.
- **AI Quiz Generation:** Produces multiple-choice quizzes based on uploaded content, enabling students to test their knowledge.

- **AI Study Plan Generation:** Generates personalized study plans according to the user's learning goals and available study time.
- **Role-Based Access Control (RBAC):** Provides different access privileges for Students and Administrators to ensure secure system management.
- **MongoDB Data Storage:** Efficiently stores user information, uploaded materials, and AI-generated learning resources using MongoDB.

Roles and Responsibilities

Student

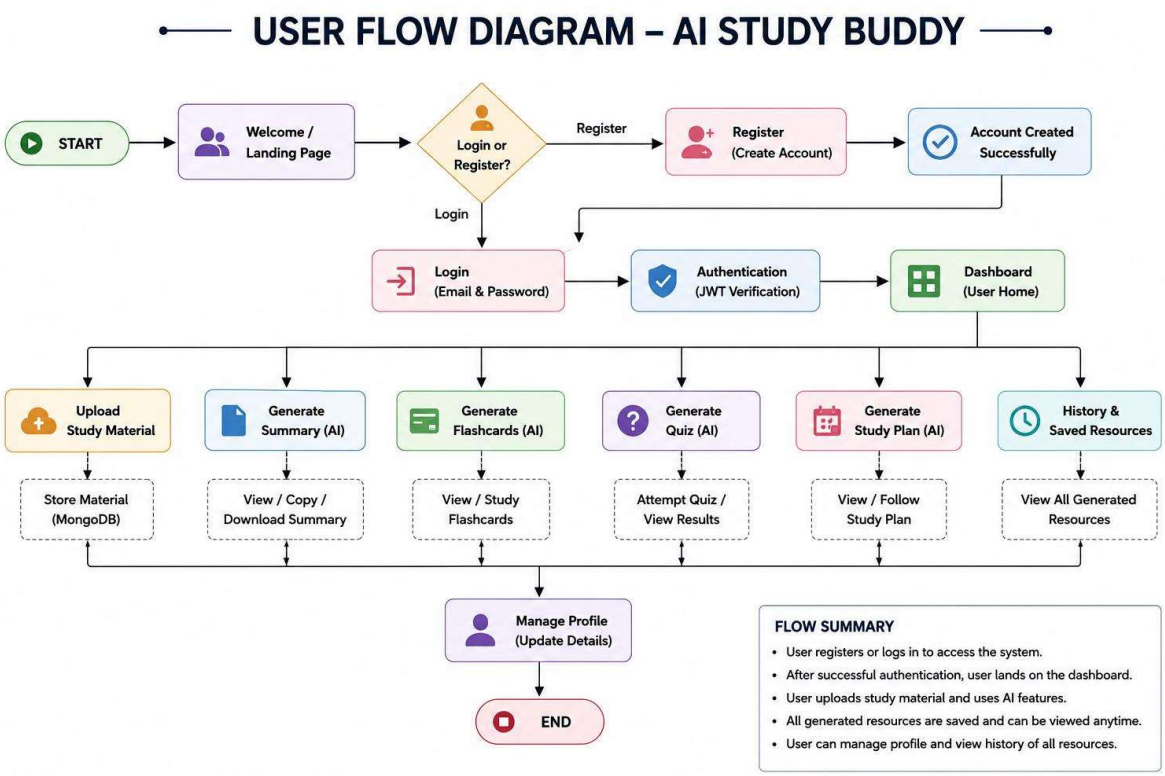
Students are the primary users of the system. They can register, log in securely, upload study materials, and utilize AI-powered features such as summary generation, flashcard creation, quiz generation, and personalized study plans. Students can also view and manage their previously generated learning resources.

Administrator

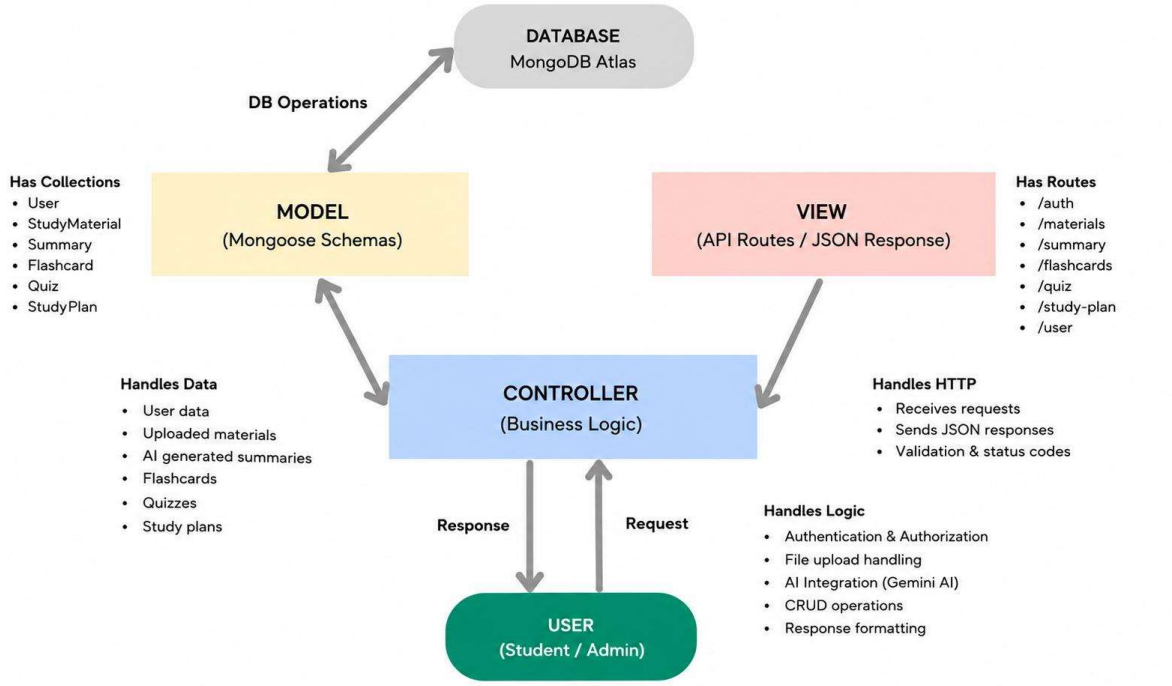
Administrators manage the overall system and ensure its smooth operation. They monitor registered users, manage user accounts, oversee application usage, maintain system integrity, and ensure that AI-powered services function correctly. Administrators also have permission to perform system-level management tasks and resolve user-related issues.

User Flow

The AI StudyBuddy application begins with the user registering or logging into the system. After successful JWT-based authentication, the user is redirected to the dashboard, where they can upload study materials and access AI-powered features. Based on the uploaded content, the system generates summaries, flashcards, quizzes, or personalized study plans using the Google Gemini API. All generated resources are displayed to the user and can be saved for future reference. Users can also manage their profile and view their previously generated learning resources through the dashboard.



MVC Architecture Pattern



The AI StudyBuddy backend follows the **Model-View-Controller (MVC)** architecture to organize the application into separate layers, making the code modular, maintainable, and scalable.

Model Layer (Data Layer)

The Model layer defines the MongoDB collections using Mongoose schemas. It is responsible for managing user information, uploaded study materials, and AI-generated learning resources while ensuring data validation and database interactions.

View Layer (API Response Layer)

Since AI StudyBuddy is a backend REST API, there is no traditional user interface in the View layer. Instead, the API returns structured JSON responses to the React frontend, allowing users to interact with the application through REST endpoints.

Controller Layer (Business Logic)

The Controller layer processes incoming API requests, validates user input, performs authentication and authorization, communicates with the database models, invokes the Google Gemini AI service when required, and returns the appropriate response to the client.

MVC Benefits

The MVC architecture separates data management, business logic, and client communication into independent components. This improves code readability, simplifies debugging and testing, and allows new features to be integrated without affecting existing modules.

Epic 2: Project Setup and Configuration

Creating the Project Folder

1. Create a new folder named **AI StudyBuddy .**
2. Open the project folder in **Visual Studio Code.**
3. Initialize the project using Node.js and install the required dependencies.

Project Initialization

Open the project folder in the VS Code terminal and execute:

```
npm init -y
```

Install the required dependencies:

```
npm install express mongoose bcryptjs jsonwebtoken cors dotenv multer @google/genai
```

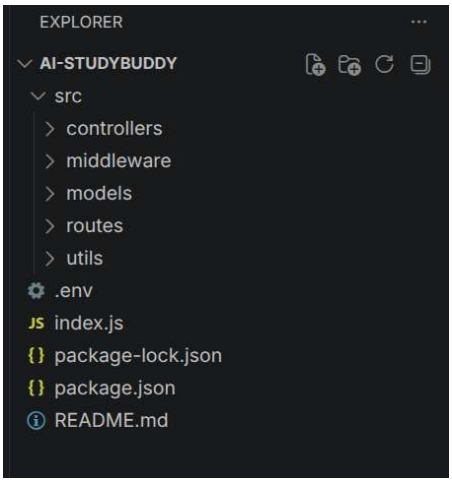
Install Nodemon for development:

```
npm install --save-dev nodemon
```

Create the Following Files

- index.js
- .env
- package.json
- .gitignore

Create the Following Folder Structure



Environment Configuration

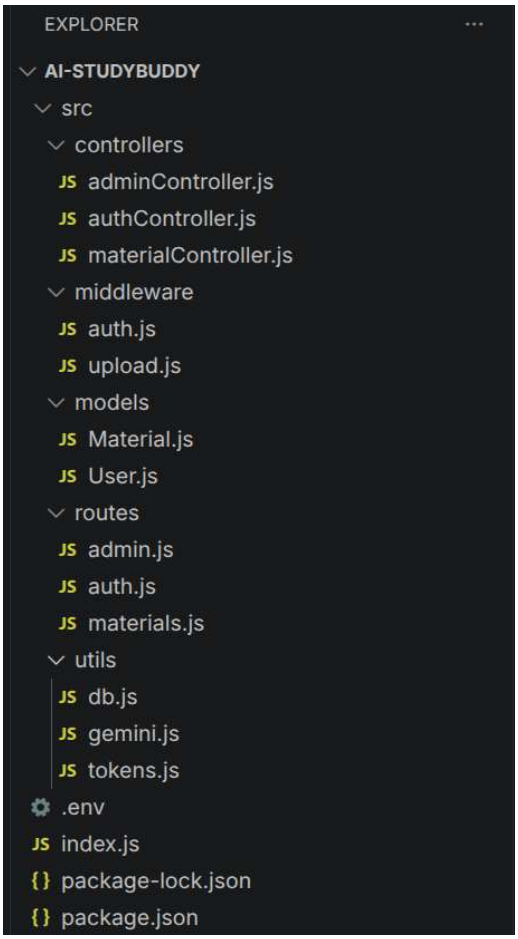
Create a **.env** file in the project root and configure the required environment variables.

PORT=5000
MONGO_URI=your_mongodb_connection_string
JWT_SECRET=your_jwt_secret
GEMINI_API_KEY=your_gemini_api_key

Epic 3: Backend Development

The AI StudyBuddy backend is developed using **Node.js**, **Express.js**, and **MongoDB**, following a modular architecture. The application separates routing, controllers, middleware, models, and utility modules to improve code maintainability and scalability. The backend exposes secure REST APIs for authentication, study material management, AI-powered content generation, and administrative operations.

Backend Structure



Core System Script Implementations

1. Application Entry Point (index.js)

The **index.js** file serves as the main entry point of the application. It initializes the Express server, loads environment variables, configures middleware, establishes the MongoDB connection, registers API routes, and starts the server after a successful database connection.



2. Database Connection (src/utils/db.js)

This module establishes a connection between the application and MongoDB using Mongoose. It ensures that the backend communicates with the database before processing any client requests.

src > utils > JS db.js > ...

```
const mongoose = require("mongoose");
const dotenv = require("dotenv");

dotenv.config();

const connectDB = async () => {
  const conn = await mongoose.connect(process.env.MONGO_URI);
  console.log(`MongoDB connected: ${conn.connection.host}`);
};

module.exports = connectDB;
```

3. Authentication Middleware (src/middleware/auth.js)

The authentication middleware verifies JWT access tokens for protected routes. It authenticates users before granting access to secured APIs and includes role-based authorization to restrict administrative operations.

src > middleware > JS auth.js > ...

```
const jwt = require("jsonwebtoken");

const protect = (req, res, next) => {
  const authHeader = req.headers.authorization;
  if (!authHeader || !authHeader.startsWith("Bearer ")) {
    return res.status(401).json({ message: "Not authenticated" });
  }

  const token = authHeader.split(" ")[1];

  try {
    const decoded = jwt.verify(token, process.env.JWT_ACCESS_SECRET);
    req.user = decoded;
    next();
  } catch {
    return res.status(401).json({ message: "Invalid or expired token" });
  }
};

const adminOnly = (req, res, next) => {
  if (req.user.role !== "admin") {
    return res.status(403).json({ message: "Admins only" });
  }
  next();
};

module.exports = { protect, adminOnly };
```

4. AI Integration (src/utils/gemini.js)

This utility integrates the backend with the **Google Gemini 2.5 Flash API**. It sends prompts to the AI model and returns generated responses for summaries, flashcards, quizzes, and study plans.



5. File Upload Middleware (src/middleware/upload.js)

The upload middleware handles study material uploads and validates incoming files before they are processed by the backend.

```
src > middleware > JS upload.js > ...
1  const multer = require("multer");
2  const path = require("path");
3
4  const storage = multer.diskStorage({
5    destination: (req, file, cb) => cb(null, "uploads/"),
6    filename: (req, file, cb) => {
7      const unique = Date.now() + "-" + Math.round(Math.random() * 1e9);
8      cb(null, unique + path.extname(file.originalname));
9    },
10 });
11
12 const fileFilter = (req, file, cb) => {
13   const allowed = [".txt", ".md", ".pdf"];
14   const ext = path.extname(file.originalname).toLowerCase();
15   if (allowed.includes(ext)) {
16     cb(null, true);
17   } else {
18     cb(new Error("Only .txt, .md, and .pdf files are allowed"));
19   }
20 };
21
22 const upload = multer({ storage, fileFilter, limits: { fileSize: 5 * 1024 * 1024 } });
23
24 module.exports = upload;
```

Backend Development Demo:

AI_Studybuddy_Backend_Demo.mkv

Epic 4: Database Configuration

The AI StudyBuddy backend uses **MongoDB** as its primary NoSQL database and **Mongoose ODM** for data modeling and schema validation. MongoDB stores user information, uploaded study materials, and AI-generated learning resources in a structured manner. Mongoose simplifies database operations by providing schemas, validation rules, and efficient query handling.

Configure MongoDB

Install Mongoose using the following command:

```
npm install mongoose
```

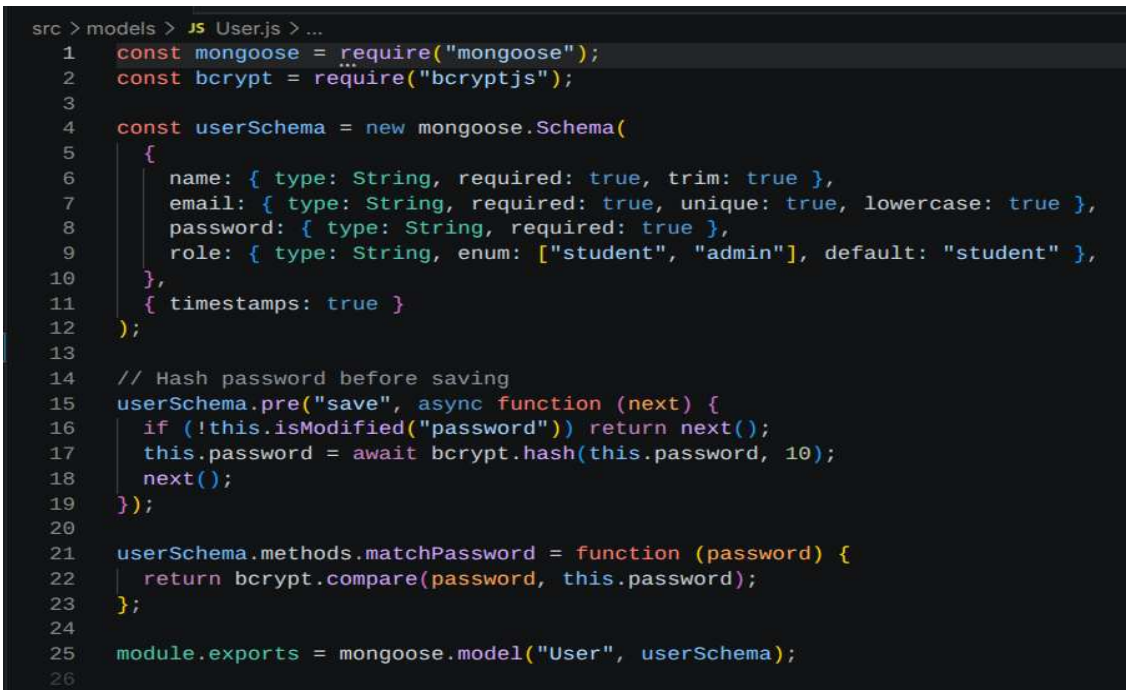
Create Database Connection

Before performing any database operations, establish a connection between the backend and MongoDB using Mongoose. The connection module is initialized during server startup to ensure reliable communication with the database.

Mongoose Schema Definitions

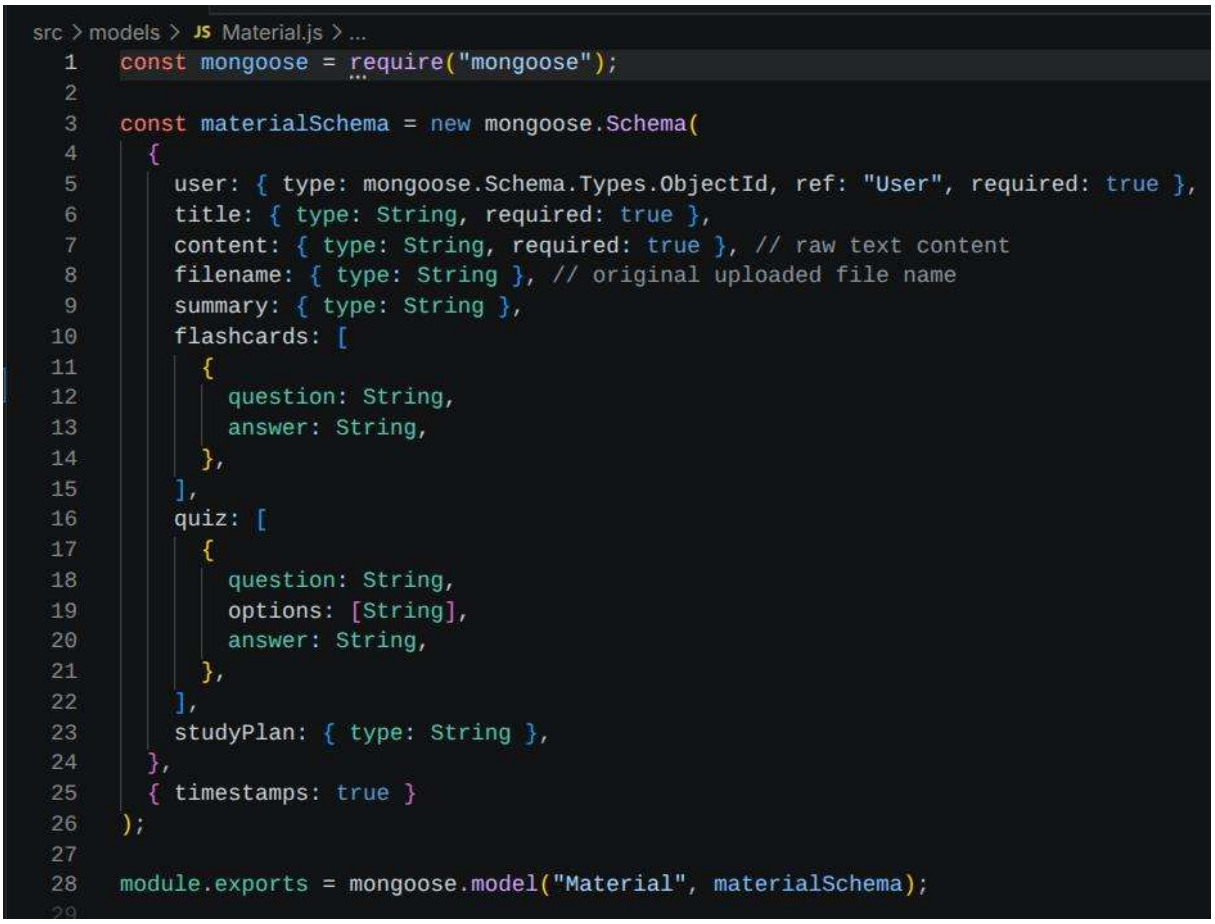
1. User Schema (`models/User.js`)

The **User** schema stores authentication and profile information for registered users. It includes fields such as name, email, password, and role. Passwords are securely encrypted before being stored in the database.



2. Study Material Schema (`models/Material.js`)

The **Study Material** schema stores the learning content uploaded by users. This content acts as the input for AI-powered features such as summaries, quizzes, flashcards, and study plans.



Database Collections

The AI StudyBuddy database consists of the following collections:

- **Users** – Stores user account and authentication details.
- **Study Materials** – Stores uploaded learning materials.

API Route Interface Endpoints

The backend exposes RESTful APIs to interact with the database and AI services.

1. Authentication APIs

Handles user registration, login, JWT authentication, and protected route access.

2. Study Material APIs

Allows users to upload, retrieve, update, and manage study materials.

3. AI Feature APIs

Provides endpoints for:

- AI Summary Generation
- AI Flashcard Generation
- AI Quiz Generation
- AI Study Plan Generation

4. Administrative APIs

Provides administrator-only endpoints for managing users and monitoring system activities.

Epic 5: Project Execution

The AI StudyBuddy backend can be executed locally using Node.js. Before running the application, ensure that MongoDB is running and the required environment variables are configured correctly. The backend exposes REST APIs that can be tested using Postman or Thunder Client.

Running the Backend Server

Install all required dependencies:

```
npm install
```

Start the development server:

```
npm start
```

After successful execution, the server connects to MongoDB and starts listening for incoming API requests.

API Testing

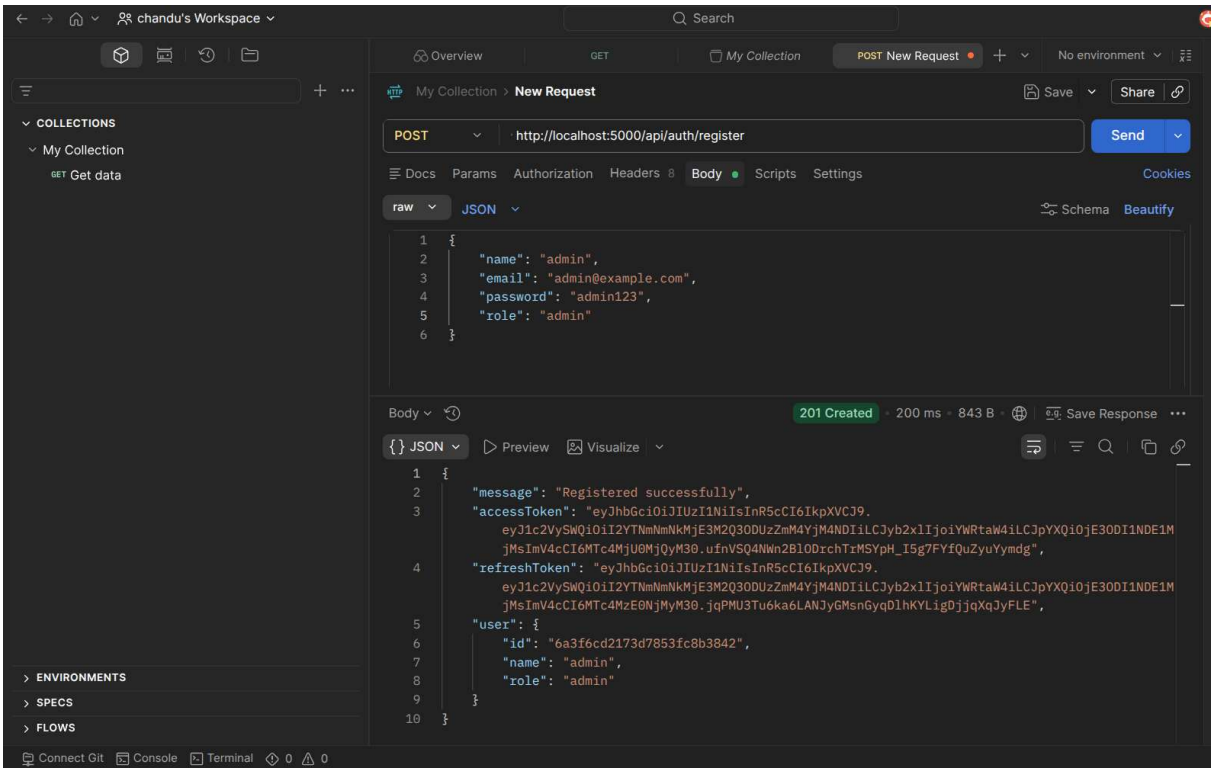
The backend APIs can be tested using **Postman**.

1. User Registration

Registers a new user by accepting the required details and securely storing the encrypted password in the MongoDB database.

Endpoint:

POST /api/auth/register

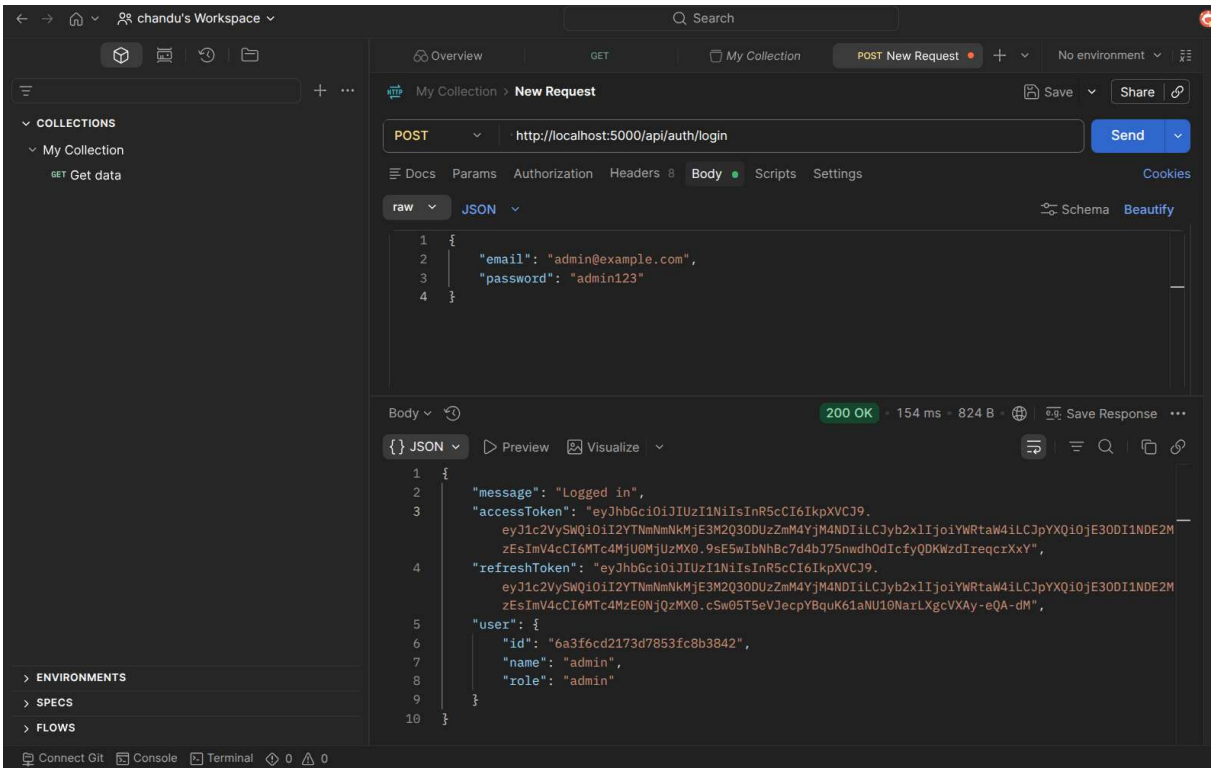


2. User Login

Authenticates the registered user and generates a JWT access token for accessing protected APIs.

Endpoint:

POST /api/auth/login

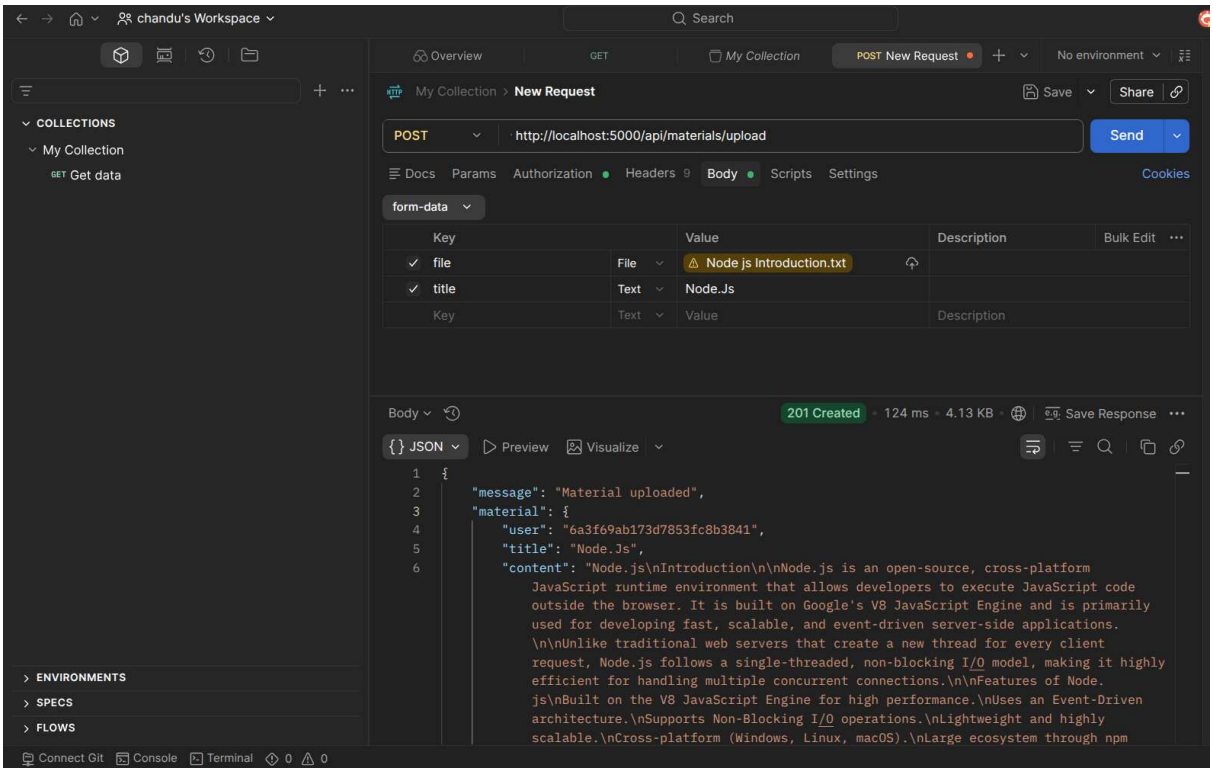


3. Upload Study Material

Allows authenticated users to upload study material that will be used as input for AI-powered content generation.

Endpoint:

POST /api/material/upload

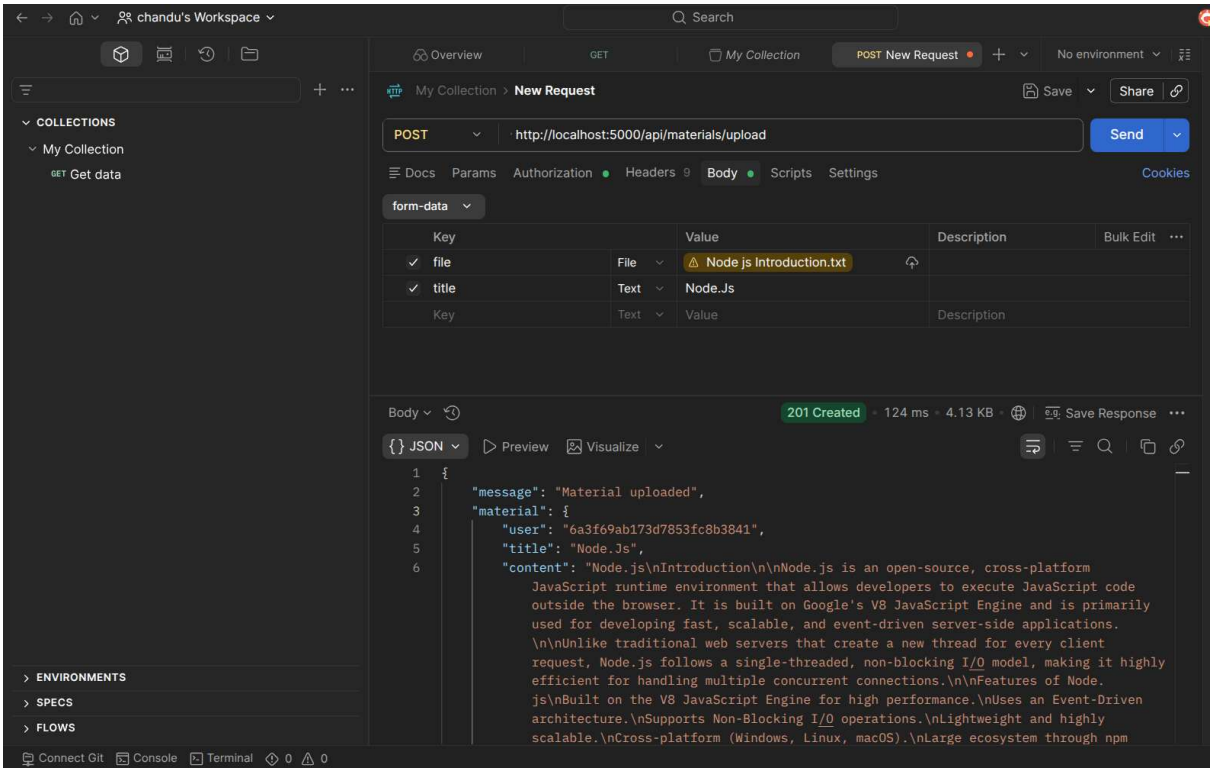


4. Generate AI Summary

Sends the uploaded study material to the Google Gemini API and generates a concise summary.

Endpoint:

POST /api/materials/:id/summarize

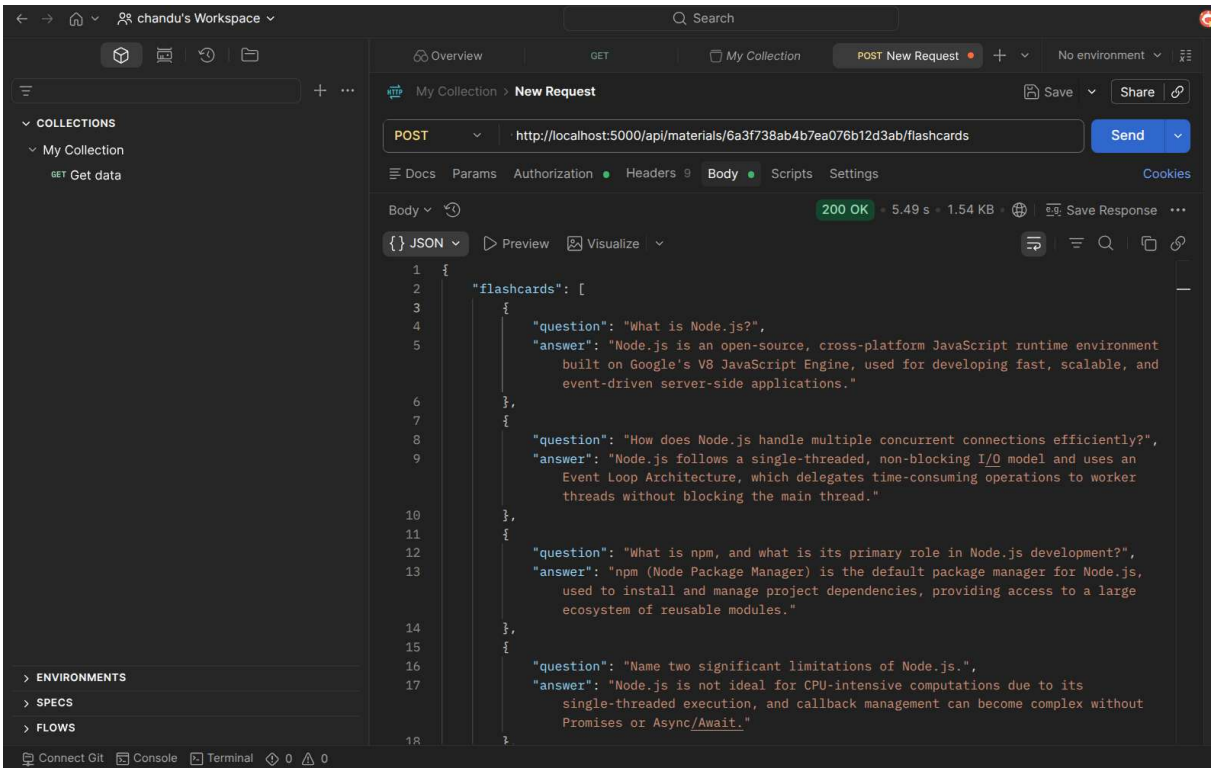


5. Generate AI Flashcards

Creates AI-generated flashcards from the uploaded study material for effective revision.

Endpoint:

POST /api/ai/flashcards

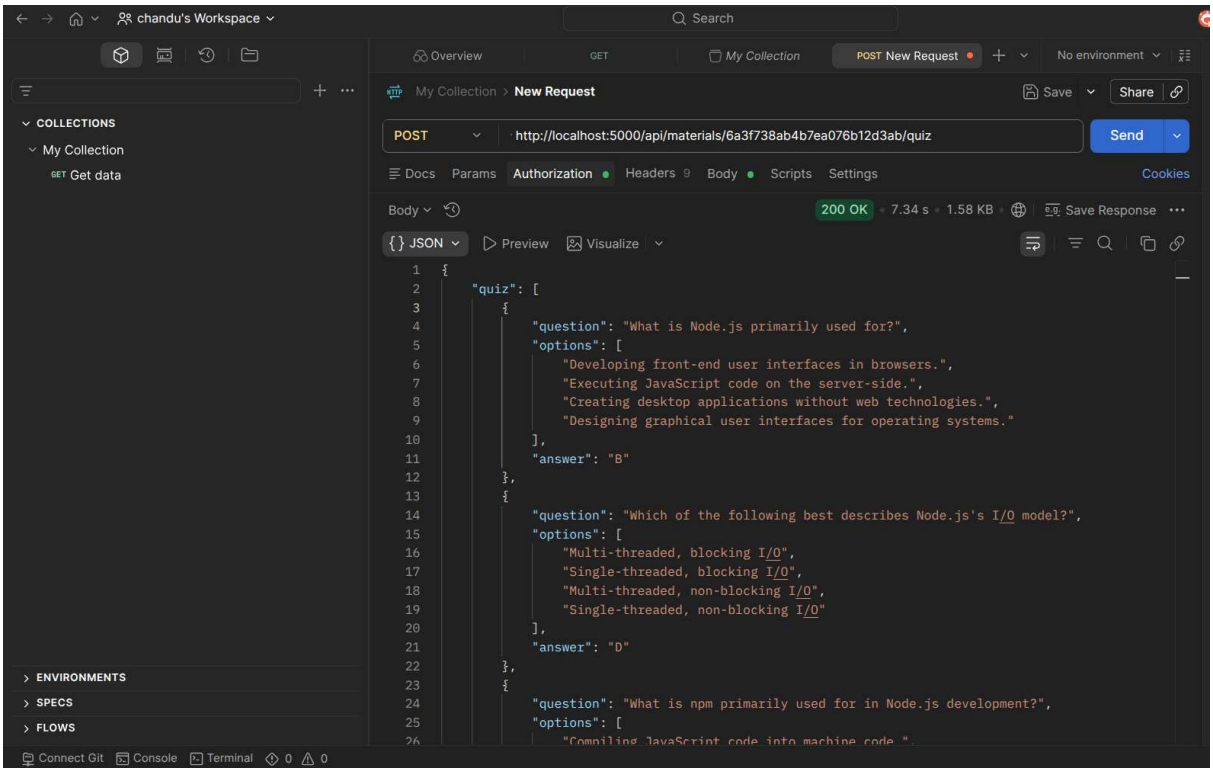


6. Generate AI Quiz

Generates multiple-choice questions from the uploaded study material to help users assess their understanding.

Endpoint:

POST /api/ai/quiz

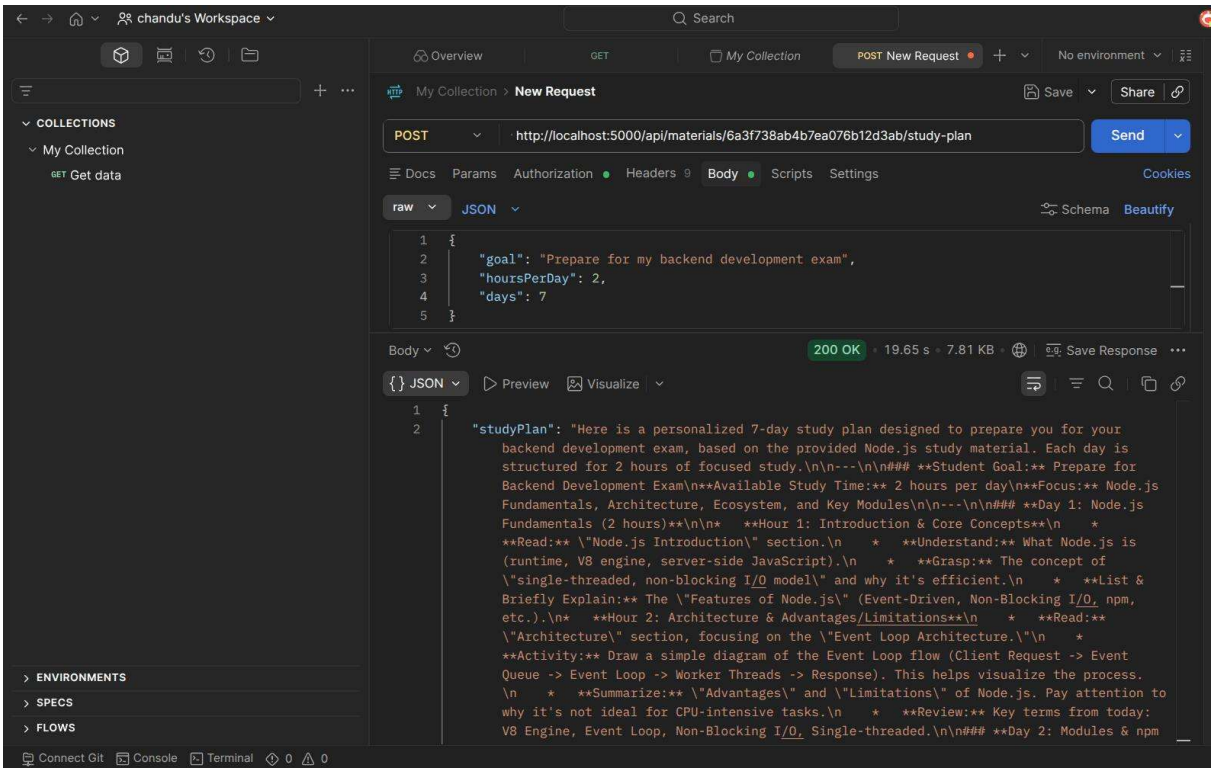


7. Generate AI Study Plan

Creates a personalized study schedule based on the user's learning goals and available study time.

Endpoint:

POST /api/ai/study-plan



API TESTING LINK : AI_Studybuddy_Execution_Demo.mkv

DEMO VIDEO LINK : https://drive.google.com/file/d/11ShkBRp-qSXogvOqFn4InZa3CQok1GFr/view?usp=drive_link

Code Drive Link : Code